Trust-Aware Memory Intelligence System

Overview

The Trust-Aware Memory Intelligence System is a multi-agent framework designed to store, manage, and explain knowledge while maintaining trust awareness.

Unlike traditional memory systems that simply store information, this system:

- Tracks confidence scores
- Detects duplicate facts
- Detects contradictions
- Maintains belief history
- Records every decision
- Provides explainable reasoning

The system processes incoming claims and continuously updates trust in stored memories based on supporting or conflicting evidence.

Problem Statement

Traditional AI memory systems suffer from several limitations:

1. They store information without tracking trust.
2. They overwrite facts when contradictory information arrives.
3. They provide little explainability.
4. They cannot show how beliefs evolve over time.
5. They do not maintain memory lifecycle states.

Example

Claim 1:

Startup A raised funding of $5M in 2021.

Claim 2:

Startup A raised funding of $8M in 2021.

Question:

Which claim should the system trust?

Most systems either overwrite the old fact or store both facts without trust management.

Our objective is to build a Trust-Aware Memory Intelligence System capable of:

- Tracking confidence
- Handling contradictions
- Preserving belief history
- Providing explainable decisions

Technology Stack

Backend

- Python 3.11

Storage

- JSON

UI

- Streamlit

Architecture

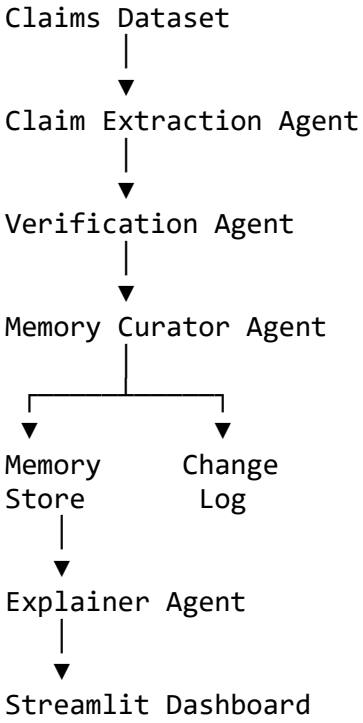
- Multi-Agent System

Project Structure

```
hackethon/
│
├── agents/
│   ├── claim_extractor.py
│   ├── verifier.py
│   ├── memory_curator.py
│   └── explainer.py
│
├── storage/
│   ├── memory_manager.py
│   ├── change_log_manager.py
│   ├── memory_store.json
│   └── change_log.json
│
├── data/
│   ├── claims.jsonl
│   └── schema.json
```

```
│
├── app.py
├── main.py
├── test_explainer.py
└── PROJECT_DOCUMENTATION.md
```

System Architecture



Agent Descriptions

1. Claim Extraction Agent

File

`agents/claim_extractor.py`

Purpose

Converts raw claims into structured facts.

Example Input

```
Startup A raised funding of $5M in 2021.
```

Output

```
{
  "subject": "Startup A",
  "predicate": "raised funding of",
  "object": "$5M in 2021"
}
```

Problem Solved

Transforms unstructured claims into structured memory records.

---

2. Verification Agent

File

`agents/verifier.py`

Purpose

Assigns an initial confidence score to every claim.

Confidence Formula

The confidence score is calculated as:

Confidence = (0.7 × Source Reliability) + (0.3 × Verification Score)

Formula

genui{“math_block_widget_always_prefetch_v2”:{“content”:“Confidence=(0.7\times SourceReliability)+(0.3\times VerificationScore)”}}

Current Implementation

```
source_reliability = claim["source_reliability"]

verification_score = 0.5
```

Why 0.7 and 0.3?

70% weight is assigned to source reliability because source credibility is considered the primary trust signal.

30% weight is assigned to verification because the system architecture reserves space for future verification models.

Why Verification Score = 0.5?

A value of 0.5 represents a neutral verification score.

The system neither strongly supports nor rejects the claim.

Example

Source Reliability:

```
0.85
```

Verification Score:

```
0.50
```

Calculation:

```
Confidence =
(0.7 × 0.85)
+
(0.3 × 0.50)

Confidence = 0.745

Rounded = 0.74
```

Problem Solved

Introduces trust scoring instead of blindly accepting information.

---

3. Memory Curator Agent

File

`agents/memory_curator.py`

Purpose

Determines how incoming claims affect memory.

The Memory Curator classifies claims into:

- NEW
- DUPLICATE
- CONTRADICTION

NEW

No matching memory exists.

Action:

- Create memory
- Add ACCEPTED log

DUPLICATE

Condition:

Same Subject

Same Predicate

Same Object

Example:

```
Startup A raised $5M
Startup A raised $5M
```

Action:

```
corroboration_count += 1

confidence += 0.03
```

Why increase confidence?

Duplicate claims provide supporting evidence.

More evidence increases trust.

CONTRADICTION

Condition:

Same Subject

Same Predicate

Different Object

Example:

```
Startup A raised $5M

Startup A raised $8M
```

Action:

```
confidence -= 0.03
```

Why decrease confidence?

Conflicting information reduces certainty.

Instead of deleting memories, the system preserves history and lowers trust.

Corroboration Count

Corroboration Count represents the amount of supporting evidence for a memory.

Example

Claim 1:

`Startup A raised $5M`

Corroboration Count:

1

Duplicate Claim:

`Startup A raised $5M`

Corroboration Count:

2

More supporting claims result in higher corroboration counts and increased trust.

Memory Store

File

`storage/memory_store.json`

Purpose

Stores persistent memory records.

Memory Structure

```
{
  "subject": "Startup A",
  "predicate": "raised funding of",
  "object": "$5M in 2021",
  "confidence": 0.62,
```

```
  "status": "active",
  "corroboration_count": 3
}
```

Problem Solved

Provides stateful memory.

Change Log

File

`storage/change_log.json`

Purpose

Tracks every memory operation.

Logged Actions

- ACCEPTED
- MERGED
- DOWNGRADED
- FORGOTTEN

Example

```
{
  "claim_id": "C004",
  "action": "DOWNGRADED",
  "reason": "Contradiction detected"
}
```

Problem Solved

Provides transparency and traceability.

Belief History

Purpose

Tracks how beliefs evolve.

Example

```
ACCEPTED (C001)

MERGED (C002)

MERGED (C003)

CONTRADICTION (C004)

CONTRADICTION (C005)
```

Problem Solved

Explains confidence evolution.

Memory Lifecycle Management

The system manages memory trust levels.

Active

Default state.

```
confidence >= 0.30
```

Under Review

```
if confidence < 0.30:
    status = "under_review"
```

Forgotten

```
if confidence < 0.20:
    status = "forgotten"
```

Problem Solved

Prevents unreliable memories from remaining trusted forever.

Explainer Agent

File

```
agents/explainer.py
```

Purpose

Generates human-readable explanations.

Example Output

```
FACT:
Startup A raised funding of $5M in 2021

CONFIDENCE:
0.62

STATUS:
active

SUPPORTED BY:
3 claims

BELIEF HISTORY:

ACCEPTED
MERGED
MERGED
CONTRADICTION
```

Problem Solved

Provides explainability.

Streamlit Dashboard

File

`app.py`

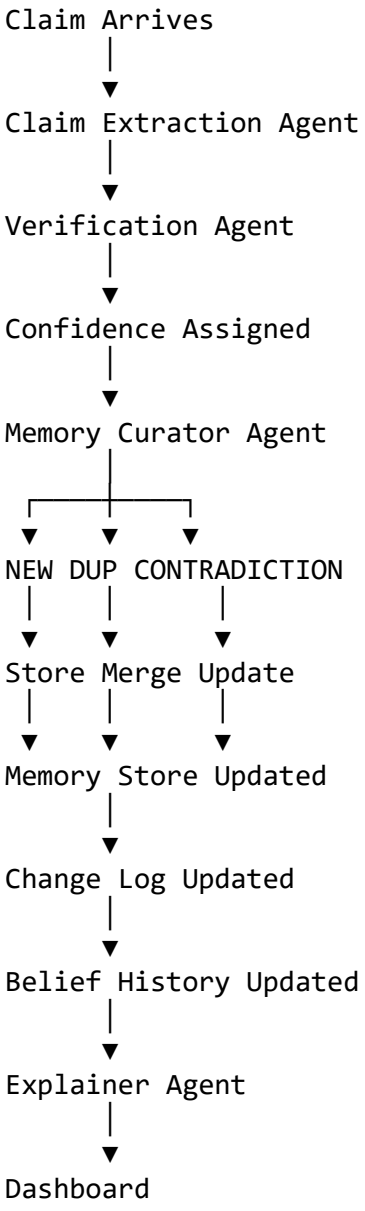
Features

- KPI Cards
- Confidence Distribution
- Search
- Memory Store
- Change Log
- Architecture View

Purpose

Provides a visual demonstration interface.

Complete Project Flow



Results

Claims Processed:

49

Unique Memories Stored:

29

Log Entries Generated:

49

Requirements Fulfilled

- ✅ Memory Store
- ✅ Change Log
- ✅ Demonstration Flow
- ✅ Stateful Memory
- ✅ Explicit Memory Logic
- ✅ Explainability
- ✅ Duplicate Detection
- ✅ Contradiction Detection
- ✅ Confidence Evolution
- ✅ Belief History
- ✅ Trust-Aware Reasoning
- ✅ Memory Lifecycle Management
- ✅ Memory Overflow Handling

Conclusion

The Trust-Aware Memory Intelligence System transforms memory from simple storage into a trust-aware reasoning framework.

Instead of merely storing information, it evaluates trust, tracks contradictions, maintains belief history, records all decisions, and provides explainable reasoning for every memory.

Future Scope: LLM Integration

Why Use an LLM?

The current prototype uses rule-based memory curation and a fixed verification score.

While this is sufficient for demonstrating trust-aware memory, it has limitations when handling:

- Semantic duplicates
- Different wording of the same fact
- Complex contradictions
- Fact verification against external knowledge

Large Language Models (LLMs) can significantly improve these capabilities.

Current Limitation

The system currently compares facts using exact matching.

Example:

Claim 1:

Startup A raised funding of $5M in 2021.

Claim 2:

Startup A raised funding of five million dollars in 2021.

Humans understand these statements mean the same thing.

The current system may treat them as different because the text is not identical.

Future LLM-Based Duplicate Detection

Current Logic

```
if (
  subject == subject
  and predicate == predicate
  and object == object
):
  DUPLICATE
```

Future Logic

LLM Prompt:

"Do these two claims represent the same fact?"

Claim A:
Startup A raised funding of $5M in 2021.

Claim B:
Startup A raised funding of five million dollars in 2021.

Expected Response:

YES

Result:

DUPLICATE

This would allow semantic duplicate detection.

Future LLM-Based Contradiction Detection

Current Logic

Same Subject

Same Predicate

Different Object

↓

CONTRADICTION

Future Logic

LLM Prompt:

"Are these claims contradictory?"

Claim A:
Startup A raised $5M in 2021.

Claim B:
Startup A raised $8M in 2021.

Expected Response:

CONTRADICTION

This enables reasoning beyond simple string matching.

Future LLM-Based Verification

Current:

verification_score = 0.5

Future:

OpenAI GPT

or

Claude

or

Llama

will verify claims.

Example Prompt:

"Evaluate the likelihood that this claim is correct and return a confidence score between 0 and 1."

Output:

{

"verification_score": 0.82

}

New Formula:

Confidence =
(0.5 × Source Reliability)
+
(0.5 × LLM Verification Score)

This would provide more intelligent trust estimation.

Future LLM-Based Explainability

Current Explainer:

Displays stored belief history.

Future Explainer:

LLM generates natural-language reasoning.

Example:

"The claim is currently trusted because it has been corroborated by three independent sources and has maintained a confidence score above the trust threshold despite receiving contradictory evidence."

This improves user understanding.

Future LangGraph Integration

LangGraph can orchestrate agent